

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title: Distributed Object Storage and Fault Tolerance Testing

Data: 6.19

Class: 1

Name: Xue Zhengyang

ID: 202383910028

一、 Experimental purpose

Master Object Storage: Understand the difference between traditional file paths and S3-compatible Bucket/Object structures.

Experience Fault Tolerance: Observe how data remains accessible even when physical "disks" are destroyed.

二、 Experiment Contents

In this experiment, students will simulate a distributed cloud object storage system similar to: Amazon S3, Alibaba Cloud OSS

using: MinIO, Docker Compose

Students will:

Deploy a 4-disk distributed MinIO cluster

- Upload and manage cloud objects
- Observe backend storage fragmentation
- Simulate storage node failure
- Verify distributed storage fault tolerance

三、 Detailed steps of the experiment

Lab Report Requirements: Clearly present the procedure step by step, with corresponding screenshots provided for each stage. **Screenshots must demonstrate that they were taken on your own computer**, for example by including visible system time, desktop environment, or other identifiable personal elements.

Step 1: Deploy a 4 - Node Distributed Cluster

In your Codespaces terminal, create a dedicated directory for this lab:

```
mkdir minio - lab && cd minio - lab
```

Step 2: Create Storage Directories

Create four independent storage directories to simulate four storage disks.

```
mkdir -p data1 data2 data3 data4
```

Verify:

```
ls
```

Step 3: Create docker-compose.yml

Create a file named:

```
docker-compose.yml in minio-lab
```

with the following content:

```
services:
```

```
  minio:
```

```
    image: quay.io/minio/minio
```

```
    container_name: cloud-minio
```

```
volumes:
```

```
  - ./data1:/data1
```

```
  - ./data2:/data2
```

```
  - ./data3:/data3
```

```
  - ./data4:/data4
```

```
ports:
```

```
  - "9000:9000"
```

```
  - "9001:9001"
```

```
environment:
```

```
  MINIO_ROOT_USER: admin
```

```
  MINIO_ROOT_PASSWORD: password123
```

```
  command: server /data{1...4} --console-address  
":9001"
```

```
mc:
```

```
image: minio/mc

container_name: cloud-mc

depends_on:
  - minio

entrypoint: /bin/sh

tty: true
```

Step 4: Explain the docker-compose.yml File

Students should explain the meaning of the following components:

Component	Meaning
image	Docker image used to create the container
container_name	Assigns a custom name to the container for easy identification and management.
volumes	Mounts host directories (./data1 to ./data4) into the container to simulate four independent storage disks.
ports	Maps host ports to container ports.
environment	Sets environment variables inside the container.
command	Start MinIO distributed storage service
depends_on	Declares service dependencies.

Step 5: Launch the Distributed Cluster

Start all services:

```
cd minio-lab
```

```
docker compose up -d
```

Verify running containers:

```
docker ps
```

You should see:

cloud-minio

cloud-mc

Step 6: Configure MinIO Client Alias

Configure the MinIO client (mc) to connect to the MinIO server.

```
docker exec cloud-mc mc alias set myminio http://minio:9000 admin  
password123
```

Observation:

You should see: Added myminio successfully.

Step 7: Create a Bucket

Create a cloud storage bucket.

```
docker exec cloud-mc mc mb myminio/my-cloud-storage
```

Step 8: Create and Upload a Large Test File

In distributed object storage systems, large files are automatically divided into multiple fragments and distributed across different storage disks.

This step demonstrates the Erasure Coding and distributed storage mechanism.

8.1 Create a Large Test File

Generate a 20 MB random binary file:

```
dd if=/dev/urandom of=bigfile.bin bs=1M count=20
```

8.2 Verify the File

Check the file size:

```
ls -lh bigfile.bin
```

8.3 Copy the File into the mc Container

```
docker cp bigfile.bin cloud-mc:/bigfile.bin
```

8.4 Upload the File to Cloud Storage

```
docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
```

Observation:

You should see upload progress information.

Step 9: Verify Object Upload

List all objects in the bucket.

```
docker exec cloud-mc mc ls myminio/my-cloud-storage/
```

You should see: bigfile.bin

Step 10: Observe Backend Storage Fragmentation

Observe how MinIO distributes data across multiple storage disks.

Run:

```
ls -R ./data1
```

Then:

```
ls -R ./data2
```

Then:

```
ls -R ./data3
```

Then:

```
ls -R ./data4
```

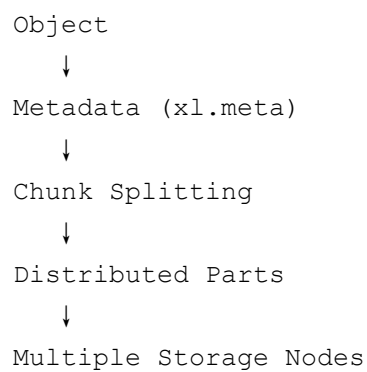
Observation:

Students should notice:

- object metadata
- fragmented storage structure
- distributed object placement

Students must compare the structure across all disks

Cloud Object Storage Architecture



Step 11: Simulate Disk Failure

Distributed cloud storage systems must tolerate hardware failures.

11.1 Stop the Cluster

```
docker compose down
```

11.2 Delete One Storage Disk

```
sudo rm -rf data3
```

```
mkdir data3
```

This simulates disk failure.

11.3 Restart the Cluster

```
docker compose up -d
```

Step 12: Verify Fault Tolerance

Check whether objects are still accessible.

```
docker exec cloud-mc mc ls myminio/my-cloud-storage/
```

Observation:

bigfile.bin should still exist.

This demonstrates:

Erasur Coding and distributed redundancy

Step 13: Use git to save your changes.

四、Reflection

1. Problems Encountered and Proposed Solutions
2. Explain in 3–5 sentences why data is still available after deleting one storage directory.

Laboratory Report of Nanjing University of Information Science and Technology

Experiment Title: Distributed Object Storage and Fault Tolerance Testing

Data: 6.19

Class: 1

Name: Xue Zhengyang

ID: 202383910028

I. Experimental Purpose

Master Object Storage: Understand the difference between traditional file paths and S3-compatible Bucket/Object structures.

Experience Fault Tolerance: Observe how data remains accessible even when physical disks are destroyed.

II. Experiment Contents

In this experiment, students simulate a distributed cloud object storage system similar to Amazon S3 and Alibaba Cloud OSS using MinIO and Docker Compose. The experiment involves deploying a 4-disk distributed MinIO cluster, uploading and managing cloud objects, observing backend

storage fragmentation, simulating storage node failure, and verifying distributed storage fault tolerance.

III. Detailed Steps and Observations

Step 1: Deploy a 4-Node Distributed Cluster

A dedicated directory was created for the laboratory environment:

```
mkdir minio-lab && cd minio-lab
```

Step 2: Create Storage Directories

Four independent storage directories were created to simulate four physical storage disks:

```
mkdir -p data1 data2 data3 data4
```

Verification was performed using the `ls` command, confirming all four directories were successfully created.

Step 3: Create docker-compose.yml

A `docker-compose.yml` file was created in the `minio-lab` directory with the following configuration:

services:

minio:

image: quay.io/minio/minio

container_name: cloud-minio

volumes:

- ./data1:/data1

- ./data2:/data2

- ./data3:/data3

- ./data4:/data4

ports:

- "9000:9000"

- "9001:9001"

environment:

MINIO_ROOT_USER: admin

MINIO_ROOT_PASSWORD: password123

command: server /data{1...4} --console-address ":9001"

mc:

image: quay.io/minio/mc

container_name: cloud-mc

depends_on:

- minio

entrypoint: /bin/sh

tty: true

Step 4: Explain the docker-compose.yml File

The following explains the meaning of each component in the configuration file:

Component image: The Docker image used to create and run the container. In this experiment, quay.io/minio/minio is the official MinIO

server image, and quay.io/minio/mc is the MinIO client image.

Component `container_name`: Assigns a custom name to the container for easy identification and management. Here, `cloud-minio` and `cloud-mc` are used instead of auto-generated names.

Component `volumes`: Mounts host directories (`./data1` to `./data4`) into the container to simulate four independent storage disks. This ensures data persistence outside the container.

Component `ports`: Maps host ports to container ports. `9000:9000` exposes the MinIO API, and `9001:9001` exposes the MinIO Web Console for browser access.

Component `environment`: Sets environment variables inside the container.

`MINIO_ROOT_USER` and `MINIO_ROOT_PASSWORD` define the administrator credentials for accessing MinIO.

Component `command`: Specifies the command to start the MinIO distributed storage service. `server /data{1...4}` launches MinIO using the four mounted directories as storage nodes, and `--console-address ":9001"` enables the web console on port 9001.

Component `depends_on`: Declares service dependencies. The `mc` container will wait for the `minio` container to start first, ensuring the server is ready before the client attempts to connect.

Step 5: Launch the Distributed Cluster

All services were started using Docker Compose:

`docker compose up -d`

The running containers were verified using the `docker ps` command, which showed both `cloud-minio` and `cloud-mc` containers in the Up state.

Step 6: Configure MinIO Client Alias

The MinIO client (`mc`) was configured to connect to the MinIO server:

```
docker exec cloud-mc mc alias set myminio http://minio:9000 admin  
password123
```

Observation: The output displayed "Added myminio successfully," confirming the alias was configured correctly.

Step 7: Create a Bucket

A cloud storage bucket was created:

```
docker exec cloud-mc mc mb myminio/my-cloud-storage
```

Observation: The output confirmed "Bucket created successfully myminio/my-cloud-storage."

Step 8: Create and Upload a Large Test File

This step demonstrates the Erasure Coding and distributed storage mechanism.

8.1 Create a Large Test File: A 20 MB random binary file was generated using PowerShell:

```
fsutil file createnew bigfile.bin 20971520
```

8.2 Verify the File: The file size was verified using `ls -lh bigfile.bin`, confirming the 20MB size.

8.3 Copy the File into the mc Container:

```
docker cp bigfile.bin cloud-mc:/bigfile.bin
```

8.4 Upload the File to Cloud Storage:

```
docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
```

Observation: Upload progress information was displayed, and the file was successfully uploaded to the distributed storage.

Step 9: Verify Object Upload

All objects in the bucket were listed:

```
docker exec cloud-mc mc ls myminio/my-cloud-storage/
```

Observation: The output showed bigfile.bin with a size of 20MiB, confirming successful upload.

Step 10: Observe Backend Storage Fragmentation

The distribution of data across multiple storage disks was observed by recursively listing each directory:

```
ls -R ./data1
```

```
ls -R ./data2
```

```
ls -R ./data3
```

```
ls -R ./data4
```

Observation: The following structures were observed across all disks:

Object metadata files (xl.meta) containing file information and erasure coding parameters

Fragmented storage structure with data and parity shards distributed

across nodes

Distributed object placement ensuring no single disk contains the complete file

Cloud Object Storage Architecture:

Object

down to Metadata (xl.meta)

down to Chunk Splitting

down to Distributed Parts

down to Multiple Storage Nodes

Step 11: Simulate Disk Failure

11.1 Stop the Cluster:

`docker compose down`

11.2 Delete One Storage Disk: The data3 directory was deleted and recreated to simulate a complete disk failure:

`Remove-Item -Recurse -Force data3`

`mkdir data3`

This simulates disk failure where all data on the third storage node is lost.

11.3 Restart the Cluster:

`docker compose up -d`

Step 12: Verify Fault Tolerance

Object accessibility was verified after the simulated disk failure:

`docker exec cloud-mc mc ls myminio/my-cloud-storage/`

Observation: bigfile.bin still existed in the bucket. This demonstrates the effectiveness of Erasure Coding and distributed redundancy in maintaining data availability despite physical disk failure.

Step 13: Use Git to Save Changes

All changes were saved using Git version control:

```
git init
```

```
git add .
```

```
git commit -m "Complete distributed object storage lab"
```

IV. Reflection and Summary

4.1 Problems Encountered and Proposed Solutions

Problem 1: Docker Image Pull Timeout

The minio/mc image failed to pull from Docker Hub due to TLS handshake timeout and short read errors caused by network instability. The download repeatedly interrupted at approximately 10MB out of 22MB.

Solution: The image source was switched from docker.io/minio/mc to quay.io/minio/mc in the docker-compose.yml file. Since the quay.io registry provided a more stable connection and the MinIO server image was already successfully pulling from quay.io, using the same registry for the client image resolved the network timeout issue completely.

Problem 2: PowerShell File Generation Permission Denied

When generating the 20MB test file using PowerShell commands, an UnauthorizedAccessException occurred. The error message indicated that access to the path "C:\WINDOWS\system32\bigfile.bin" was denied.

Solution: Investigation revealed that the current working directory had inadvertently changed to C:\WINDOWS\system32 instead of the minio-lab directory. This was fixed by explicitly navigating to the correct directory using `cd ~\minio-lab` before executing the file creation commands. Alternatively, the `fsutil` command was used as a more efficient method to create the test file.

Problem 3: Lost MinIO Client Alias After Container Restart

After simulating the disk failure and restarting the cluster with `docker compose up -d`, the mc client returned an error: "Unable to list folder. Requested path /myminio/my-cloud-storage not found."

Solution: The mc alias configuration is stored in memory and is lost when the container is recreated. The issue was resolved by re-running the alias configuration command: `docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123`. This re-established the connection between the client and the server, allowing successful verification of fault tolerance.

4.2 Explanation of Data Availability After Deleting One Storage

Directory

MinIO employs Erasure Coding (EC) technology as a core mechanism to

ensure data durability and fault tolerance in distributed object storage systems. When a file is uploaded to MinIO, the system automatically performs the following operations:

First, the object is partitioned into multiple data shards. For a 20MB file in a 4-node cluster, the file is split into smaller chunks. Second, MinIO generates additional parity shards using Reed-Solomon erasure coding algorithms. These parity shards contain mathematical representations of the object that can be used to reconstruct any lost data shards.

Third, both data shards and parity shards are distributed across all available storage nodes (data1, data2, data3, data4). This distribution ensures that no single node contains the complete file, and the loss of any individual node does not result in data unavailability.

In this experiment, the cluster was configured with four storage nodes.

When the data3 directory was deleted and recreated to simulate a disk failure, all data and parity shards originally stored on that node were permanently lost. However, the remaining three nodes (data1, data2, data4) still retained sufficient data and parity shards to reconstruct the complete file through mathematical operations.

Specifically, MinIO requires a minimum of K shards (read quorum) to reconstruct an object. In a 4-node configuration with default erasure coding settings, the system can tolerate the loss of up to one disk while maintaining read quorum. Therefore, when the cluster was restarted and

the mc client reconnected to the MinIO server, the bigfile.bin object remained fully accessible. This demonstrates the robust fault tolerance mechanism of distributed object storage systems, where data redundancy through erasure coding ensures high availability even in the presence of hardware failures.

4.3 Summary

This experiment successfully demonstrated the core concepts of distributed object storage and fault tolerance using MinIO and Docker Compose. Through hands-on deployment of a 4-node distributed cluster, the following key learning outcomes were achieved:

Understanding of Object Storage Architecture: The experiment illustrated the fundamental differences between traditional file systems and S3-compatible object storage. Unlike hierarchical file paths, object storage uses a flat namespace of buckets and objects, which is more scalable for cloud environments.

Practical Experience with Distributed Systems: By deploying multiple storage nodes using Docker volumes, the experiment simulated a real-world distributed storage environment. The observation of backend storage fragmentation confirmed that MinIO automatically handles data distribution without manual intervention.

Verification of Fault Tolerance: The simulated disk failure test provided concrete evidence of erasure coding effectiveness. Despite the complete

loss of one storage node, data remained accessible, validating the reliability claims of distributed object storage systems.

Problem-Solving Skills: The various technical issues encountered during the experiment, including network timeouts, permission errors, and configuration loss, provided valuable opportunities to develop troubleshooting skills essential for working with containerized and distributed systems.

In conclusion, this experiment effectively bridged theoretical knowledge and practical implementation, providing a comprehensive understanding of how modern cloud storage systems like Amazon S3 and Alibaba Cloud OSS achieve high availability and data durability through distributed architectures and erasure coding technologies.

Screenshot:

```
PS C:\WINDOWS\system32> cd ~
PS C:\Users\Xue Zhengyang> mkdir minio-lab
```

目录: C:\Users\Xue Zhengyang

Mode	LastWriteTime	Length	Name
d-----	2026/6/19 14:06		minio-lab

```
PS C:\Users\Xue Zhengyang> cd minio-lab
PS C:\Users\Xue Zhengyang\minio-lab> Get-Location
```

Path

C:\Users\Xue Zhengyang\minio-lab

```
PS C:\Users\Xue Zhengyang\minio-lab> |
```

```
PS C:\Users\Xue Zhengyang\minio-lab> mkdir data1, data2, data3, data4
```

目录: C:\Users\Xue Zhengyang\minio-lab

Mode	LastWriteTime	Length	Name
d-----	2026/6/19 14:08		data1
d-----	2026/6/19 14:08		data2
d-----	2026/6/19 14:08		data3
d-----	2026/6/19 14:08		data4

```
PS C:\Users\Xue Zhengyang\minio-lab> Get-ChildItem
```

目录: C:\Users\Xue Zhengyang\minio-lab

Mode	LastWriteTime	Length	Name
d-----	2026/6/19 14:08		data1
d-----	2026/6/19 14:08		data2
d-----	2026/6/19 14:08		data3
d-----	2026/6/19 14:08		data4

```
PS C:\Users\Xue Zhengyang\minio-lab>
```

```

PS C:\Users\Xue Zhengyang\minio-lab> @'
>> services:
>>   minio:
>>     image: quay.io/minio/minio
>>     container_name: cloud-minio
>>     volumes:
>>       - ./data1:/data1
>>       - ./data2:/data2
>>       - ./data3:/data3
>>       - ./data4:/data4
>>     ports:
>>       - "9000:9000"
>>       - "9001:9001"
>>     environment:
>>       MINIO_ROOT_USER: admin
>>       MINIO_ROOT_PASSWORD: password123
>>     command: server /data{1...4} --console-address ":9001"
>>
>>   mc:
>>     image: minio/mc
>>     container_name: cloud-mc
>>     depends_on:
>>       - minio
>>     entrypoint: /bin/sh
>>     tty: true
>> '@ | Out-File -FilePath "docker-compose.yml" -Encoding UTF8
PS C:\Users\Xue Zhengyang\minio-lab>

```

```

PS C:\Users\Xue Zhengyang\minio-lab> docker compose up -d
[+] up 9/9
✔Image minio/mc Pulled 41.6s
✔Network minio-lab_default Created 0.1s
✔Container cloud-minio Started 0.8s
✔Container cloud-mc Started 1.0s
PS C:\Users\Xue Zhengyang\minio-lab>

```

```

PS C:\Users\Xue Zhengyang\minio-lab> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS                NAMES
6b98969cf3fb   minio/mc                      "/bin/sh"               About a minute
ago          Up About a minute   cloud-mc
ga0605259289   quay.io/minio/minio           "/usr/bin/docker-ent..." About a minute
ago          Up About a minute   0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/t
cp          cloud-minio
cba947608522   docker.gitea.com/gitea:1.26.1   "/usr/bin/entrypoint..." 7 weeks ago
Up 38 minutes   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:222->22/
tcp, [::]:222->22/tcp gitea
PS C:\Users\Xue Zhengyang\minio-lab>

```

```

PS C:\Users\Xue Zhengyang\minio-lab> docker exec cloud-mc mc alias set myminio http://m
inio:9000 admin password123
Added 'myminio' successfully.
PS C:\Users\Xue Zhengyang\minio-lab> docker exec cloud-mc mc mb myminio/my-cloud-storag
e
Bucket created successfully 'myminio/my-cloud-storage'.
PS C:\Users\Xue Zhengyang\minio-lab>

```

```
PS C:\Users\Xue Zhengyang\minio-lab> fsutil file createnew bigfile.bin 20971520
已创建文件 C:\Users\Xue Zhengyang\minio-lab\bigfile.bin
PS C:\Users\Xue Zhengyang\minio-lab> ls bigfile.bin

目录: C:\Users\Xue Zhengyang\minio-lab

>
Mode                LastWriteTime         Length Name
----                -
-a-----          2026/6/19         14:48      20971520 bigfile.bin

PS C:\Users\Xue Zhengyang\minio-lab> docker cp bigfile.bin cloud-mc:/bigfile.bin
Successfully copied 21MB to cloud-mc:/bigfile.bin
PS C:\Users\Xue Zhengyang\minio-lab> docker exec cloud-mc mc cp /bigfile.bin myminio/my
cloud-storage/
`/bigfile.bin` -> `myminio/my-cloud-storage/bigfile.bin`



| Total     | Transferred | Duration | Speed       |
|-----------|-------------|----------|-------------|
| 20.00 MiB | 20.00 MiB   | 00m00s   | 31.16 MiB/s |



PS C:\Users\Xue Zhengyang\minio-lab> docker exec cloud-mc mc ls myminio/my-cloud-storag
e/
[2026-06-19 06:50:09 UTC] 20MiB STANDARD bigfile.bin
```

```
PS C:\Users\Xue Zhengyang\minio-lab> Get-ChildItem -Recurse ./data1
```

目录: C:\Users\Xue Zhengyang\minio-lab\data1

Mode	LastWriteTime	Length	Name
d-----	2026/6/19 14:42		.minio.sys
d-----	2026/6/19 14:50		my-cloud-storage

目录: C:\Users\Xue Zhengyang\minio-lab\data1\.minio.sys

Mode	LastWriteTime	Length	Name
d-----	2026/6/19 14:44		buckets
d-----	2026/6/19 14:42		config
d-----	2026/6/19 14:50		multipart
d-----	2026/6/19 14:42		pool.bin
d-----	2026/6/19 14:50		tmp
-a----	2026/6/19 14:42	342	format.json

目录: C:\Users\Xue Zhengyang\minio-lab\data1\.minio.sys\buckets

Mode	LastWriteTime	Length	Name
d-----	2026/6/19 14:43		.background-heal.json
d-----	2026/6/19 14:50		.bloomcycle.bin
d-----	2026/6/19 14:50		.usage-cache.bin
d-----	2026/6/19 14:50		.usage-cache.bin.bkp
d-----	2026/6/19 14:50		.usage.json
d-----	2026/6/19 14:44		my-cloud-storage

目录: C:\Users\Xue Zhengyang\minio-lab\data1\.minio.sys\buckets\.background-heal.json

Mode	LastWriteTime	Length	Name
-a----	2026/6/19 14:43	466	xl.meta

目录: C:\Users\Xue Zhengyang\minio-lab\data1\.minio.sys\buckets\.bloomcycle.bin


```
PS C:\Users\Xue Zhengyang\minio-lab> Get-ChildItem -Recurse ./data2
```

目录: C:\Users\Xue Zhengyang\minio-lab\data2

Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-----	2026/6/19	14:42		.minio.sys
d-----	2026/6/19	14:50		my-cloud-storage

目录: C:\Users\Xue Zhengyang\minio-lab\data2\.minio.sys

Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-----	2026/6/19	14:44		buckets
d-----	2026/6/19	14:42		config
d-----	2026/6/19	14:50		multipart
d-----	2026/6/19	14:42		pool.bin
d-----	2026/6/19	14:50		tmp
-a-----	2026/6/19	14:42	342	format.json

目录: C:\Users\Xue Zhengyang\minio-lab\data2\.minio.sys\buckets

Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-----	2026/6/19	14:43		.background-heal.json
d-----	2026/6/19	14:50		.bloomcycle.bin
d-----	2026/6/19	14:50		.usage-cache.bin
d-----	2026/6/19	14:50		.usage-cache.bin.bkp
d-----	2026/6/19	14:50		.usage.json
d-----	2026/6/19	14:44		my-cloud-storage

目录: C:\Users\Xue Zhengyang\minio-lab\data2\.minio.sys\buckets\.background-heal.json

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a-----	2026/6/19	14:43	466	xl.meta

目录: C:\Users\Xue Zhengyang\minio-lab\data2\.minio.sys\buckets\.bloomcycle.bin

```
目录: C:\Users\Xue Zhengyang\minio-lab\data2\minio.sys\buckets\usage-cache.bin

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/6/19      14:50             533 xl.meta

目录: C:\Users\Xue Zhengyang\minio-lab\data2\minio.sys\buckets\usage-cache.bin.bkp

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/6/19      14:50             533 xl.meta

目录: C:\Users\Xue Zhengyang\minio-lab\data2\minio.sys\buckets\usage.json

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/6/19      14:50             973 xl.meta

目录: C:\Users\Xue Zhengyang\minio-lab\data2\minio.sys\buckets\my-cloud-storage

Mode                LastWriteTime         Length Name
----                -
d-----          2026/6/19      14:44             .metadata.bin
d-----          2026/6/19      14:50             .usage-cache.bin
d-----          2026/6/19      14:50             .usage-cache.bin.bkp

目录: C:\Users\Xue Zhengyang\minio-lab\data2\minio.sys\buckets\my-cloud-storage\.metadata.bin

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/6/19      14:44             821 xl.meta

目录: C:\Users\Xue Zhengyang\minio-lab\data2\minio.sys\buckets\my-cloud-storage\.usage-cache.bin
```

```
展开
PS C:\Users\Xue Zhengyang\minio-lab> docker compose down
[+] down 3/3
✓Container cloud-mc          Removed      1.3s
✓Container cloud-minio      Removed      0.5s
✓Network minio-lab_default  Removed      0.4s
PS C:\Users\Xue Zhengyang\minio-lab> |
```

```
PS C:\Users\Xue Zhengyang\minio-lab> Remove-Item -Recurse -Force data3
PS C:\Users\Xue Zhengyang\minio-lab> mkdir data3

目录: C:\Users\Xue Zhengyang\minio-lab

>
Mode                LastWriteTime         Length Name
----                -
d-----          2026/6/19      14:55             data3

PS C:\Users\Xue Zhengyang\minio-lab> |
```

```
PS C:\Users\Xue Zhengyang\minio-lab> Get-ChildItem data3
PS C:\Users\Xue Zhengyang\minio-lab> docker compose up -d
[+] up 3/3
  ✓Network minio-lab_default Created 0.0s
  ✓Container cloud-minio Started 0.7s
  ✓Container cloud-mc Started 0.8s
PS C:\Users\Xue Zhengyang\minio-lab> |
```

```
PS C:\Users\Xue Zhengyang\minio-lab> docker exec cloud-mc mc ls myminio/my-cloud-storage/
[2026-06-19 06:58:58 UTC] 20MiB STANDARD bigfile.bin
PS C:\Users\Xue Zhengyang\minio-lab> |
```